

Labwork 4: 2D Threads Configuration

Nguyen Duy Anh Tung

October 1, 2026

0.1 Introduction

In Labwork 4, we extend the RGB-to-grayscale conversion by migrating from a 1D grid layout to a 2-dimensional thread and block hierarchy[cite: 4]. The goal is to preserve the native 2D spatial representation of images, evaluate the speedup gained across various 2D block shapes, examine hardware constraints, and solve classroom exercises on thread scheduling[cite: 4].

0.2 Implementation

Unlike the 1D flattened array approach used in Labwork 3, the image is passed directly in its 2D matrix shape $(H, W, 3)$ [cite: 4]:

- **CPU Implementation:** Iterates sequentially through image height (y) and width (x) using nested loops to compute grayscale intensity:

$$g = \lfloor (R + G + B)/3 \rfloor$$

- **GPU Implementation:** Implements a 2D CUDA kernel where each thread calculates its 2D global coordinate:

$$x = \text{threadIdx.x} + \text{blockIdx.x} \times \text{blockDim.x}$$

$$y = \text{threadIdx.y} + \text{blockIdx.y} \times \text{blockDim.y}$$

A boundary check `if x < w and y < h` ensures threads beyond image borders do not perform invalid memory accesses[cite: 4].

0.3 Experimental Results

Testing was conducted on an image containing 199,875 pixels. The sequential CPU execution time was **0.2080 s** (208.0 ms). GPU execution times are averaged over 5 iterations.

Block Size	Threads/Block	Grid Size	GPU Time (ms)	Speedup
(4, 4)	16	(94, 134)	0.20	1016.3×
(8, 8)	64	(47, 67)	0.16	1329.8×
(16, 16)	256	(24, 34)	0.15	1377.7×
(32, 16)	512	(12, 34)	0.15	1393.5×
(32, 32)	1024	(12, 17)	0.16	1316.6×

Table 1: Performance comparison across different 2D block dimensions

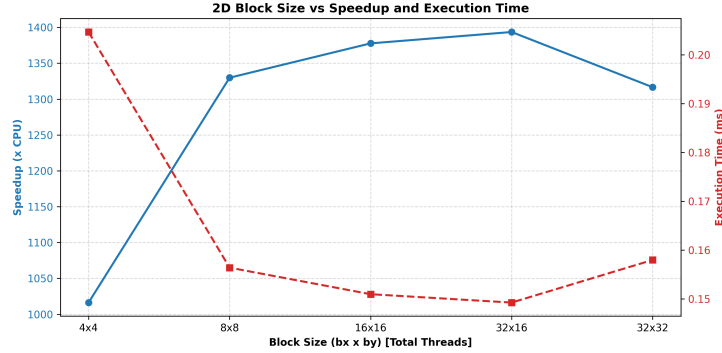


Figure 1: 2D Block Size vs Speedup over CPU

0.4 Hardware Limit and Crash Analysis

When attempting configurations such as (64, 32) (2048 threads) or (64, 64) (4096 threads), the kernel execution crashes with:

```
numba.cuda.cudadrv.driver.CudaAPIError: [1] CUDA_ERROR_INVALID_VALUE
```

This crash occurs because NVIDIA GPU architecture strictly enforces a physical limit of **1024 threads per block** (`MAX_THREADS_PER_BLOCK = 1024`)[cite: 4]. Any block launch requesting a total thread count exceeding 1024 is rejected at runtime by the CUDA driver.

0.5 Exercises & Discussion

- **Speedup:** The 2D CUDA implementation achieves massive acceleration, delivering up to $1393.5\times$ speedup over the sequential CPU baseline.
- **Occupancy & Performance:** The (4, 4) block (16 threads) yields lower performance (0.20 ms) because it is smaller than the hardware warp size (32 threads)[cite: 4], leaving execution units underutilized. Peak efficiency is reached at (32, 16) (512 threads, 0.15 ms), achieving optimal warp scheduling and SM occupancy. At the hardware maximum (32, 32) (1024 threads)[cite: 4], latency slightly increases to 0.16 ms due to register pressure and block-scheduling overhead.
- **Exercise 1:** Best configuration is 16×16 [cite: 4]. $32 \times 32 = 1024$ exceeds the 512 threads/block limit[cite: 4]. $8 \times 8 = 64$ threads only yields $8 \times 64 = 512$ threads on the SM due to the 8 blocks/SM limit (50% occupancy)[cite: 4]. $16 \times 16 = 256$ threads allows 4 blocks/SM, maximizing SM capacity at 1024 threads (100% occupancy)[cite: 4].

- **Exercise 2:** Configuration with the most threads in the SM is **512 threads/block**[cite: 4]. With a ceiling of 4 blocks/SM, $3 \times 512 = 1536$ threads fully saturates the SM limit without violating block constraints[cite: 4].
- **Exercise 3:** A vector length of 2,000 with 512 threads/block requires $\lceil 2000/512 \rceil = 4$ blocks[cite: 4]. Total threads in the grid = $4 \times 512 = 2048$ threads[cite: 4].